

Assignment 2: Multi-Client Chat Application using TCP and UDP Sockets

Simiyon Vinscent Samuel L

Register Number: 3122247001062

Submission Date: August 17, 2026

GitHub Repository: <https://github.com/blackscythe123/PCN-assign>

Aim

To build a small client-server system in C that lets a student look up their academic record by roll number – once over TCP, returning the full profile, and once over UDP, returning just enough to confirm the record exists – both reading from the same `students.csv`. The point of the exercise is really to feel the practical difference between a connection-oriented, reliable transport and a connectionless one for the same basic lookup task, not the lookup itself.

Reading the Student Data

Both servers keep this deliberately simple: on startup they open `students.csv`, skip the header line, and split each remaining line on commas straight into an array of `Student` structs held in memory. Nothing fancier than `fgets()` + `strtok()` – no CSV library, no error recovery beyond bailing out if the file itself cannot be opened. Since the array is written once at startup and only ever read afterwards, the TCP server's worker threads can search it concurrently without needing a mutex.

Listing 1: `load_students()` – shared parsing logic in both `tcp_server.c` and `udp_server.c`

```
1 char line[256];
2 fgets(line, sizeof(line), fp); /* skip the header row */
3
4 while (fgets(line, sizeof(line), fp) != NULL) {
5     line[strcspn(line, "\r\n")] = '\0';
6     if (strlen(line) == 0) continue;
7
8     Student s;
9     char *token = strtok(line, ",");
10    strncpy(s.roll, token, sizeof(s.roll) - 1); s.roll[sizeof(s.roll) - 1] = '\0';
11    token = strtok(NULL, ","); strncpy(s.name, token, sizeof(s.name) - 1); s.name[sizeof(s.name) - 1] = '\0';
12    token = strtok(NULL, ","); strncpy(s.dept, token, sizeof(s.dept) - 1); s.dept[sizeof(s.dept) - 1] = '\0';
13    token = strtok(NULL, ","); strncpy(s.sem, token, sizeof(s.sem) - 1); s.sem[sizeof(s.sem) - 1] = '\0';
14    token = strtok(NULL, ","); strncpy(s.cgpa, token, sizeof(s.cgpa) - 1); s.cgpa[sizeof(s.cgpa) - 1] = '\0';
15
16    students[student_count++] = s;
17 }
```

Part 1(a): TCP Client–Server – Full Student Profile

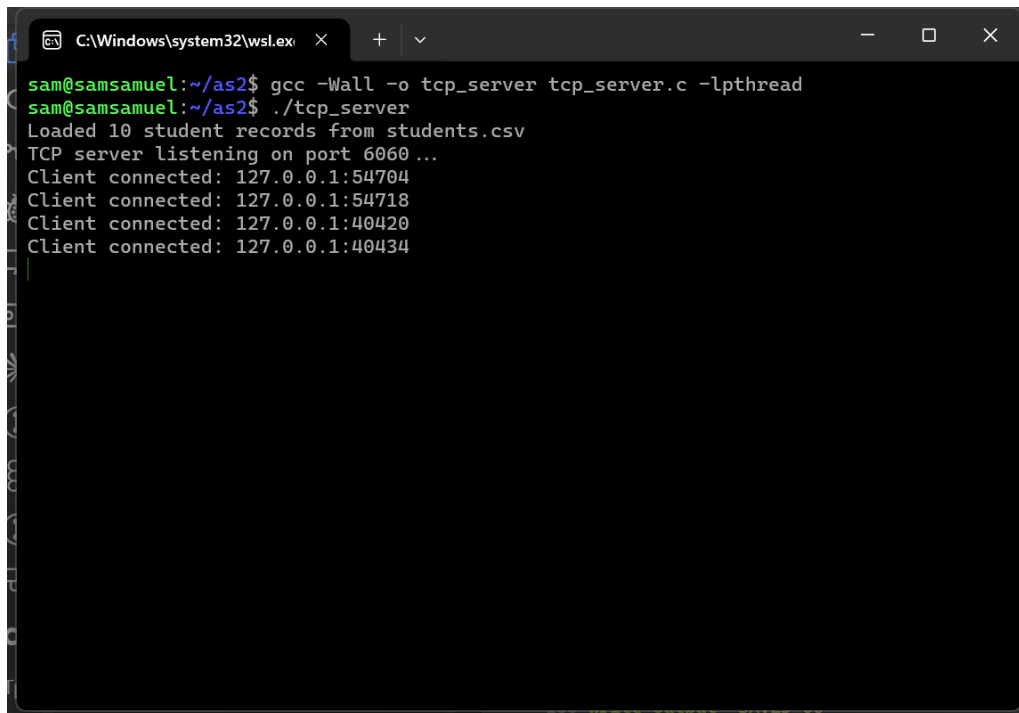
`tcp_server.c` listens on port 6060. Every accepted connection is handed off to its own pthread (`handle_client`), so multiple clients are genuinely served in parallel rather than queued behind a single `accept()` loop – confirmed in Figure 3, where two clients launched moments apart both get their correct, independent replies. Each thread reads one roll number off its socket, does a linear search of the in-memory array, and sends back either the full record or `Student Record Not Found`, then closes its own connection.

Listing 2: Excerpt of `tcp_server.c` – accepting a client and handing it off to a worker thread

```
1 int *client_fd = malloc(sizeof(int));
2 *client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &
   client_len);
3
4 if (*client_fd < 0) {
5     perror("accept");
6     free(client_fd);
7     continue;
8 }
9
10 printf("Client connected: %s:%d\n",
11        inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));
12 fflush(stdout);
13
14 pthread_t tid;
15 pthread_create(&tid, NULL, handle_client, client_fd);
16 pthread_detach(tid); /* thread cleans itself up when done, no need to
   join */
```

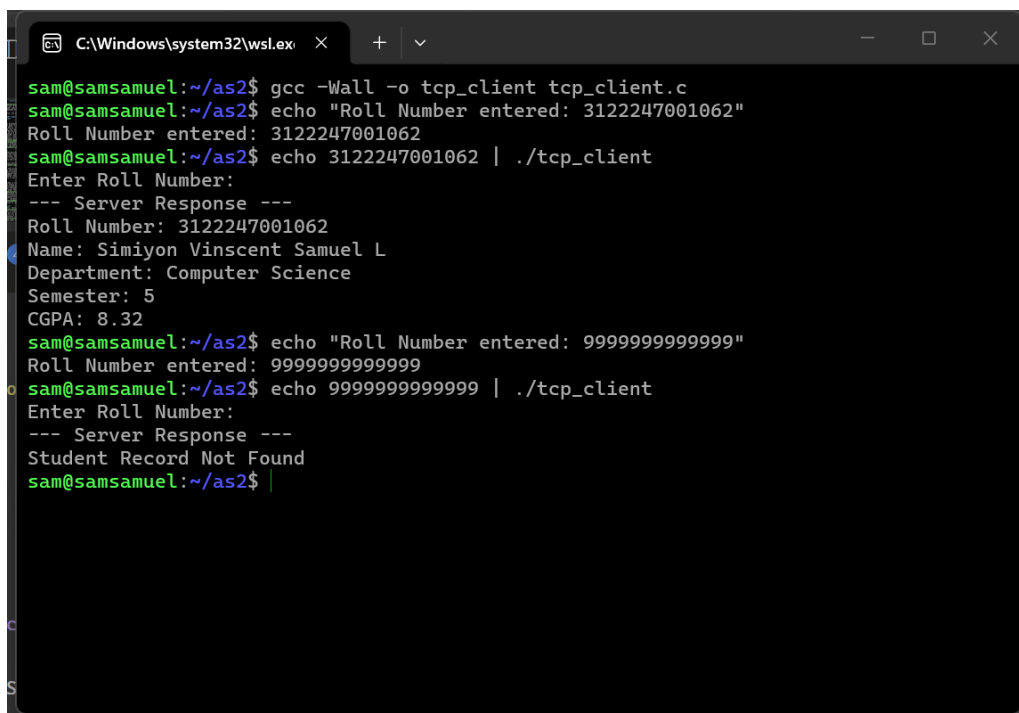
Listing 3: Excerpt of `tcp_server.c` – `handle_client()`: looking up the roll number and building the full-profile response

```
1 for (int i = 0; i < student_count; i++) {
2     if (strcmp(students[i].roll, buf) == 0) {
3         snprintf(response, sizeof(response),
4                  "Roll Number: %s\nName: %s\nDepartment: %s\nSemester: %s\n
5                  nCGPA: %s\n",
6                  students[i].roll, students[i].name, students[i].dept,
7                  students[i].sem, students[i].cgpa);
8         found = 1;
9         break;
10    }
11 }
12
13 if (!found) {
14     snprintf(response, sizeof(response), "Student Record Not Found\n");
15 }
```

A terminal window titled 'C:\Windows\system32\wsl.exe' showing the execution of a TCP server. The user 'sam@samsamuel' is in the directory '~/.as2'. They compile 'tcp_server.c' with 'gcc -Wall -o tcp_server tcp_server.c -lpthread' and then run './tcp_server'. The server loads 10 student records from 'students.csv' and listens on port 6060. It then receives four client connections from 127.0.0.1 with ports 54704, 54718, 40420, and 40434.

```
C:\Windows\system32\wsl.exe x + v
sam@samsamuel:~/.as2$ gcc -Wall -o tcp_server tcp_server.c -lpthread
sam@samsamuel:~/.as2$ ./tcp_server
Loaded 10 student records from students.csv
TCP server listening on port 6060 ...
Client connected: 127.0.0.1:54704
Client connected: 127.0.0.1:54718
Client connected: 127.0.0.1:40420
Client connected: 127.0.0.1:40434
```

Figure 1: TCP server terminal, accumulated log: a found lookup, a not-found lookup, and two near-simultaneous client connections handled by separate threads.

A terminal window titled 'C:\Windows\system32\wsl.exe' showing the execution of a TCP client. The user 'sam@samsamuel' is in the directory '~/.as2'. They compile 'tcp_client.c' with 'gcc -Wall -o tcp_client tcp_client.c'. They then run two instances of the client. The first instance enters the roll number '3122247001062' and receives a full student profile response. The second instance enters the roll number '999999999999' and receives a 'Student Record Not Found' response.

```
C:\Windows\system32\wsl.exe x + v
sam@samsamuel:~/.as2$ gcc -Wall -o tcp_client tcp_client.c
sam@samsamuel:~/.as2$ echo "Roll Number entered: 3122247001062"
Roll Number entered: 3122247001062
sam@samsamuel:~/.as2$ echo 3122247001062 | ./tcp_client
Enter Roll Number:
--- Server Response ---
Roll Number: 3122247001062
Name: Simiyon Vinscent Samuel L
Department: Computer Science
Semester: 5
CGPA: 8.32
sam@samsamuel:~/.as2$ echo "Roll Number entered: 999999999999"
Roll Number entered: 999999999999
sam@samsamuel:~/.as2$ echo 999999999999 | ./tcp_client
Enter Roll Number:
--- Server Response ---
Student Record Not Found
sam@samsamuel:~/.as2$
```

Figure 2: TCP client run twice in sequence – first an existing roll number returning the full profile, then a roll number not in students.csv.

```

C:\Windows\system32\wsl.exe
sam@samsamuel:~/as2$ echo "Launching two clients at once: 3122247001064 and 3122247001068"
Launching two clients at once: 3122247001064 and 3122247001068
sam@samsamuel:~/as2$ (echo 3122247001064 | ./tcp_client &) & (echo 3122247001068 | ./tcp_client &); wait
Enter Roll Number:
--- Server Response ---
Roll Number: 3122247001068
Name: Ramya Devi K
Department: Computer Science
Semester: 5
CGPA: 8.45
Enter Roll Number:
--- Server Response ---
Roll Number: 3122247001064
Name: Lakshmi Priya V
Department: Computer Science
Semester: 5
CGPA: 9.01
sam@samsamuel:~/as2$

```

Figure 3: Two `tcp_client` processes launched together against the same running server; both receive their own correct reply, proving the `pthread` concurrency actually works.

Part 1(b): UDP Client–Server – Verify Student Record

`udp_server.c` listens on port 6061 but never calls `listen()` or `accept()` – there is no connection to accept, just a single `recvfrom()/sendto()` loop handling one datagram at a time. Because the whole exchange fits in one loop iteration and the reply is intentionally tiny (roll number and name only, per the 1(b) spec), one thread is enough; there is no per-client state to hand off anywhere.

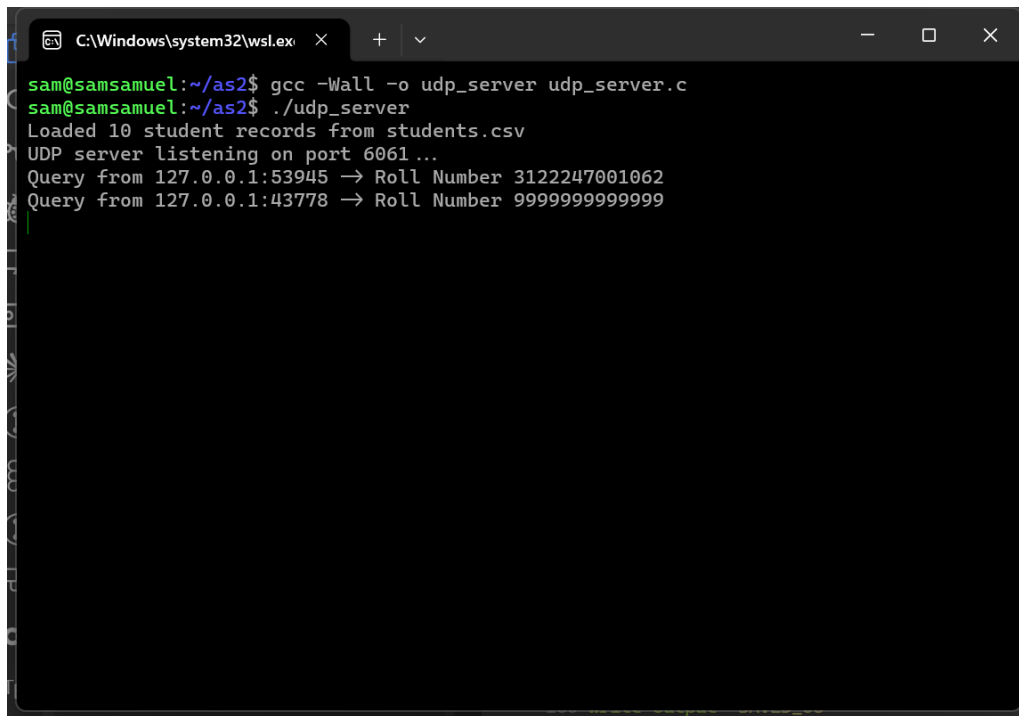
Listing 4: Excerpt of `udp_server.c` – the main `recvfrom()/sendto()` loop

```

1 while (1) {
2     int n = recvfrom(sock, buf, sizeof(buf) - 1, 0,
3                     (struct sockaddr *)&client_addr, &client_len);
4     if (n <= 0) continue;
5     buf[n] = '\0';
6     buf[strcspn(buf, "\r\n")] = '\0';
7
8     char response[BUF_SIZE];
9     int found = 0;
10
11     for (int i = 0; i < student_count; i++) {
12         if (strcmp(students[i].roll, buf) == 0) {
13             snprintf(response, sizeof(response), "Roll Number: %s\nName
14                     : %s\n",
15                     students[i].roll, students[i].name);
16             found = 1;
17             break;
18         }
19     }

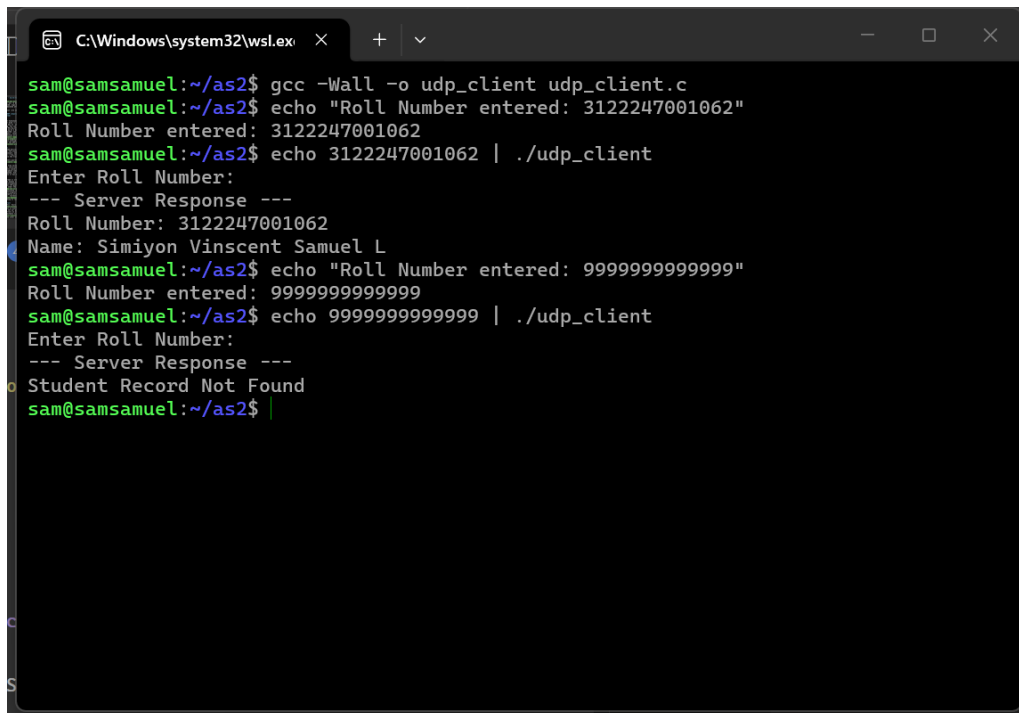
```

```
20     if (!found) {  
21         snprintf(response, sizeof(response), "Student Record Not Found\  
22             n");  
23     }  
24     sendto(sock, response, strlen(response), 0,  
25         (struct sockaddr *)&client_addr, client_len);  
26 }
```

A terminal window titled 'C:\Windows\system32\wsl.exe' showing the execution of a UDP server program. The user 'sam@samsamuel' is in the directory '~/.as2'. The commands executed are 'gcc -Wall -o udp_server udp_server.c' and './udp_server'. The output shows that 10 student records were loaded from 'students.csv' and the server is listening on port 6061. Two queries are shown: one from 127.0.0.1:53945 returning roll number 3122247001062, and another from 127.0.0.1:43778 returning roll number 9999999999999.

```
C:\Windows\system32\wsl.exe x + v  
sam@samsamuel:~/as2$ gcc -Wall -o udp_server udp_server.c  
sam@samsamuel:~/as2$ ./udp_server  
Loaded 10 student records from students.csv  
UDP server listening on port 6061 ...  
Query from 127.0.0.1:53945 → Roll Number 3122247001062  
Query from 127.0.0.1:43778 → Roll Number 9999999999999
```

Figure 4: UDP server terminal log, showing incoming roll-number queries as they arrive.

A terminal window titled 'C:\Windows\system32\wsl.exe' showing a series of commands and outputs. The user 'sam' is in the directory '~/.as2'. The first command is 'gcc -Wall -o udp_client udp_client.c'. The second is 'echo "Roll Number entered: 3122247001062"', which outputs 'Roll Number entered: 3122247001062'. The third command is 'echo 3122247001062 | ./udp_client', which prompts 'Enter Roll Number:' and then shows the 'Server Response' as 'Roll Number: 3122247001062' and 'Name: Simiyon Vinscent Samuel L'. The fourth command is 'echo "Roll Number entered: 999999999999"', which outputs 'Roll Number entered: 999999999999'. The fifth command is 'echo 999999999999 | ./udp_client', which prompts 'Enter Roll Number:' and then shows the 'Server Response' as 'Student Record Not Found'.

```
C:\Windows\system32\wsl.exe x + v
sam@samsamuel:~/as2$ gcc -Wall -o udp_client udp_client.c
sam@samsamuel:~/as2$ echo "Roll Number entered: 3122247001062"
Roll Number entered: 3122247001062
sam@samsamuel:~/as2$ echo 3122247001062 | ./udp_client
Enter Roll Number:
--- Server Response ---
Roll Number: 3122247001062
Name: Simiyon Vinscent Samuel L
sam@samsamuel:~/as2$ echo "Roll Number entered: 999999999999"
Roll Number entered: 999999999999
sam@samsamuel:~/as2$ echo 999999999999 | ./udp_client
Enter Roll Number:
--- Server Response ---
Student Record Not Found
sam@samsamuel:~/as2$
```

Figure 5: UDP client run twice – an existing roll number returning just Roll Number and Name, then a roll number not in students.csv.

Part 1c: TCP vs UDP Comparison

Aspect	TCP (Part 1a)	UDP (Part 1b)
Connection	Explicit: <code>socket</code> → <code>bind</code> → <code>listen</code> → <code>accept</code> on the server, <code>connect()</code> on the client. Each client shows up as a distinct “Client connected” line in the server log.	None. <code>udp_server</code> just <code>bind()</code> s and waits; the client fires one <code>sendto()</code> with no setup step, and there is nothing like “Client connected” in its log.
Reliability	Guaranteed by the kernel’s TCP stack – every request in testing got an answer, and neither program has to handle a lost packet because the transport already does.	No guarantee at all. <code>udp_client.c</code> calls <code>recvfrom()</code> once and just prints “No response from server.” if nothing comes back – no retry, no timeout.
Ordering	Byte stream is ordered by the protocol (moot here since each exchange is one request/response on a fresh connection, but true in general).	Datagrams can in principle arrive out of order or duplicated; irrelevant for a single-request-single-reply exchange but a real property of the protocol.
Error Recovery	<code>connect()</code> fails loudly and immediately if the server is not listening (<code>perror</code> + <code>exit</code>) – easy to detect and debug.	<code>sendto()</code> “succeeds” even if nobody is listening; the client only finds out indirectly when <code>recvfrom()</code> never returns. With no timeout coded here, an unreachable server would hang the client.
Communication	Stream-oriented, connection-scoped, one pthread per client – proven genuinely concurrent in Figure 3, not just serialized by accident.	Message-oriented, one <code>sendto/recvfrom</code> pair per exchange, stateless – a single thread in a loop is enough since there is no connection to hand off.

TCP is heavier (handshake, per-client thread, connection state) but guarantees the client gets a correct, in-order, complete profile every time. UDP is lighter and faster for the one-shot “just confirm this roll number is valid” case in 1(b), at the cost of no delivery guarantee – an acceptable trade here since it is a verify-only lookup, not a full profile fetch.

Learning Outcome

- Got a TCP server with a real pthread-per-client design working, and actually proved the concurrency by hitting it with two clients at once instead of just assuming it from the code.
- Saw how much simpler a UDP server is to write – no accept/connect, no per-client thread – but also how little protection that gives you: nothing in this lab’s code would even

notice if a datagram went missing.

- Parsing a CSV by hand with `fgets()/strtok()` was fiddlier than expected (trailing newlines, the header row eating a real record if you forget to skip it) – a good reminder that “just read the file” is rarely as trivial as it sounds.